

LAPORAN PRAKTIKUM PENGOLAHAN CITRA DIGITAL



Nama : Jonathan Steven Sitorus
NIM : 202431091
Kelas : G
Nama Dosen : Rizqia Cahyaningtyas, ST., M.Kom
No. Bangku :
Nama Asisten :
1. Muhammad Caisar Gemilang Pratama
2. Dzaki Devsi Pratama

**LABORATORIUM COMPUTER NETWORK
INSTITUT TEKNOLOGI PLN
2026**

BAB III PEMBAHASAN KODE PROGRAM

3.1 Cell 1: Impor Pustaka

Kode pada cell pertama adalah sebagai berikut:

```
In [2]: #202431091
        #Jonathan Steven Sitorus

import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Cell pertama berfungsi sebagai tahap inisialisasi program dengan melakukan impor terhadap tiga pustaka yang diperlukan. Perintah `import cv2` mengimpor pustaka OpenCV yang akan digunakan untuk membaca file gambar dari sistem, melakukan konversi format warna, serta menerapkan operasi filtering pada citra. Perintah `import numpy as np` mengimpor pustaka NumPy dengan alias `np`, yang diperlukan untuk membuat matriks kernel secara manual menggunakan fungsi `np.array()`. Terakhir, perintah `import matplotlib.pyplot as plt` mengimpor modul `pyplot` dari Matplotlib dengan alias `plt`, yang akan digunakan sepanjang program untuk menampilkan citra hasil pemrosesan dalam bentuk visual. Ketiga pustaka ini merupakan fondasi dari hampir seluruh pekerjaan pengolahan citra berbasis Python.

3.2 Cell 2: Membaca dan Menampilkan Gambar Asli

Kode pada cell kedua adalah sebagai berikut:

```
In [21]: #202431091
        #Jonathan Steven Sitorus

# Membaca gambar asli
img_original = cv2.imread('KAI.jpg.jpeg')
img_rgb = cv2.cvtColor(img_original, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(img_rgb)
plt.title('Gambar Asli')
plt.axis('off')
```

Out[21]: (-0.5, 294.5, 123.5, -0.5)



Cell kedua bertugas untuk membaca file gambar dari direktori kerja dan menampilkannya sebagai referensi visual awal. Fungsi `cv2.imread('KAI.jpg.jpeg')` membaca file gambar dengan nama tersebut dan menyimpannya sebagai array NumPy bertipe integer dalam variabel `img_original`. Perlu diperhatikan bahwa OpenCV secara default membaca citra dalam format BGR (Blue, Green, Red), bukan RGB yang umum digunakan. Oleh karena itu, langkah konversi warna menggunakan

cv2.cvtColor(img_original, cv2.COLOR_BGR2RGB) sangat penting dilakukan agar warna yang ditampilkan oleh Matplotlib sesuai dengan kondisi asli gambar.

Selanjutnya, plt.figure(figsize=(8, 6)) membuat sebuah kanvas baru dengan ukuran 8x6 inci. Fungsi plt.imshow(img_rgb) menampilkan array citra pada kanvas tersebut, plt.title('Gambar Asli') memberi judul pada tampilan, dan plt.axis('off') menyembunyikan sumbu koordinat agar tampilan citra lebih bersih dan tidak terganggu oleh informasi sumbu yang tidak relevan.

3.3 Cell 3: Operasi Konvolusi dengan Sharpening Kernel

Kode pada cell ketiga adalah sebagai berikut:

```
In [22]: #202431091
#Jonathan Steven Sitorus

# Mendefinisikan kernel penajaman (Sharpening Kernel)
kernel_sharpen = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]])

# Menerapkan konvolusi menggunakan filter2D
img_convolution = cv2.filter2D(img_gray, -1, kernel_sharpen)

# Menampilkan hasil konvolusi
plt.figure(figsize=(5,5))
plt.imshow(img_convolution, cmap='gray')
plt.title('Hasil Operasi Konvolusi (Sharpening)')
plt.axis('off')
plt.show()
```



Cell ketiga

merupakan inti dari demonstrasi operasi konvolusi. Program terlebih dahulu mendefinisikan sebuah kernel penajaman berukuran 3x3 menggunakan np.array(). Kernel ini memiliki nilai +5 di posisi tengah dan nilai -1 pada posisi atas, bawah, kiri, dan kanan, serta nilai 0 di sudut-sudut diagonal. Struktur kernel semacam ini dikenal sebagai Laplacian-based sharpening filter.

Cara kerja kernel ini adalah sebagai berikut: nilai piksel pusat dikalikan dengan 5, lalu dikurangi dengan nilai keempat tetangganya. Apabila suatu piksel memiliki nilai yang mirip dengan tetangganya (area yang datar), maka hasilnya akan mendekati nilai asli piksel tersebut. Sebaliknya, apabila terdapat perbedaan nilai yang signifikan antar piksel (area tepi atau detail), maka perbedaan tersebut akan diperkuat sehingga menghasilkan tepi yang lebih tajam.

Fungsi cv2.filter2D(img_gray, -1, kernel_sharpen) menerapkan operasi konvolusi antara citra grayscale (img_gray) dengan kernel yang telah didefinisikan. Argumen -1 menandakan bahwa tipe data keluaran sama dengan tipe data masukan. Hasil konvolusi disimpan dalam variabel img_convolution, kemudian ditampilkan menggunakan Matplotlib dengan colormap 'gray' karena citra yang diproses adalah citra grayscale.

3.4 Cell 4: Operasi Ketetanggaan (Average dan Median Filtering)

Kode pada cell keempat adalah sebagai berikut:

```
In [23]: #202431091
#Jonathan Steven Sitorus

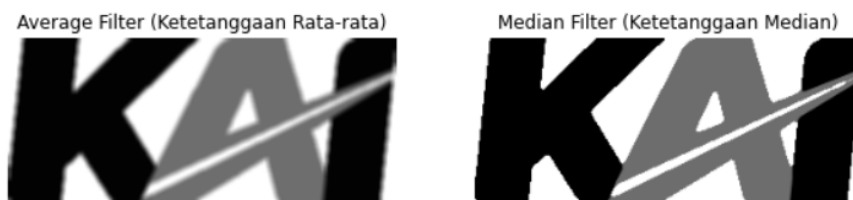
# 1. Average Filtering (Ketetanggaan dengan meratakan nilai)
img_average = cv2.blur(img_gray, (5,5))

# 2. Median Filtering (Ketetanggaan dengan nilai tengah)
img_median = cv2.medianBlur(img_gray, 5)

# Menampilkan perbandingan hasil metode ketetanggaan
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(img_average, cmap='gray')
axes[0].set_title('Average Filter (Ketetanggaan Rata-rata)')
axes[0].axis('off')

axes[1].imshow(img_median, cmap='gray')
axes[1].set_title('Median Filter (Ketetanggaan Median)')
axes[1].axis('off')

plt.show()
```



In []:

In []:

Cell keempat mengimplementasikan dua jenis operasi ketetanggaan dan menampilkan hasilnya secara berdampingan untuk memudahkan perbandingan. Pertama, fungsi `cv2.blur(img_gray, (5,5))` menerapkan Average Filtering dengan ukuran jendela 5x5 piksel. Fungsi ini menghitung rata-rata aritmetika dari seluruh 25 piksel dalam jendela tersebut untuk menghasilkan nilai piksel baru. Parameter (5,5) menentukan dimensi jendela yang digunakan; semakin besar ukuran jendela, maka citra yang dihasilkan akan semakin halus namun juga semakin kehilangan detail.

Kedua, fungsi `cv2.medianBlur(img_gray, 5)` menerapkan Median Filtering dengan ukuran jendela 5x5. Berbeda dengan average filtering, fungsi ini mengurutkan seluruh nilai piksel dalam jendela dan mengambil nilai yang berada di posisi tengah sebagai nilai piksel keluaran. Karakteristik unik dari median filter adalah kemampuannya dalam mempertahankan ketajaman tepi (edge-preserving) sambil tetap efektif mengurangi noise, terutama noise jenis garam dan merica (salt-and-pepper noise).

Untuk menampilkan perbandingan, digunakan `plt.subplots(1, 2, figsize=(10, 5))` yang menciptakan sebuah figure dengan dua subplot bersebelahan. Variabel `axes` yang dikembalikan adalah array yang berisi dua objek sumbu, sehingga `axes[0]` merujuk pada subplot kiri untuk menampilkan hasil average filter, dan `axes[1]` merujuk pada subplot kanan untuk menampilkan hasil median filter. Pendekatan ini memungkinkan perbandingan visual yang langsung dan intuitif antara kedua metode.

BAB IV HASIL DAN ANALISIS

4.1 Hasil Pelaksanaan Program

Program berhasil dijalankan secara keseluruhan di dalam lingkungan Jupyter Notebook. Masing-masing cell menghasilkan keluaran visual berupa citra yang ditampilkan langsung di bawah cell yang bersangkutan. Cell kedua menampilkan gambar asli dalam format warna RGB sebagai referensi. Cell ketiga menampilkan citra hasil operasi konvolusi dengan sharpening kernel, sedangkan cell keempat menampilkan dua citra secara berdampingan, yaitu hasil average filter dan median filter.

4.2 Analisis Hasil Konvolusi

Hasil dari operasi konvolusi menggunakan sharpening kernel menunjukkan peningkatan ketajaman yang jelas pada area-area yang mengandung detail dan transisi nilai piksel yang tajam, seperti tepian objek dan garis-garis pada gambar. Hal ini sesuai dengan sifat matematis dari kernel yang digunakan, yaitu memperkuat perbedaan intensitas lokal. Pada area yang datar atau homogen, nilai piksel relatif tidak banyak berubah karena tidak terdapat perbedaan signifikan yang dapat diperkuat oleh kernel.

4.3 Analisis Hasil Operasi Ketetangaan

Perbandingan antara hasil average filter dan median filter memberikan wawasan yang menarik. Kedua metode berhasil menghaluskan citra dari gambar aslinya, namun dengan karakteristik yang berbeda. Hasil average filter menunjukkan efek penghalusan yang merata namun cenderung membuat tepi-tepi objek menjadi kurang tajam karena proses perataan nilai ikut memengaruhi piksel-piksel di area transisi.

Sementara itu, hasil median filter terlihat mampu mempertahankan ketajaman tepi dengan lebih baik. Ini disebabkan karena operasi median tidak sensitif terhadap nilai-nilai ekstrem yang biasanya muncul pada area tepi objek. Oleh karena itu, median filter secara umum lebih direkomendasikan apabila tujuan pemrosesan adalah untuk mengurangi noise tanpa mengorbankan kejelasan tepi objek dalam citra.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan pelaksanaan praktikum pengolahan citra digital dengan topik operasi konvolusi dan ketetanggaan, dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, operasi konvolusi menggunakan sharpening kernel terbukti efektif dalam meningkatkan ketajaman detail dan tepi objek pada citra. Kernel penajaman bekerja dengan cara memperkuat perbedaan nilai intensitas lokal antar piksel yang bertetangga, sehingga informasi tepi menjadi lebih menonjol secara visual.

Kedua, operasi ketetanggaan melalui average filtering dan median filtering keduanya mampu menghaluskan citra, namun dengan karakteristik yang berbeda. Average filtering menghasilkan penghalusan yang lebih seragam namun berpotensi mengurangi ketajaman tepi, sedangkan median filtering lebih unggul dalam mempertahankan tepi objek sekaligus mengurangi noise.

Ketiga, Python dengan kombinasi pustaka OpenCV, NumPy, dan Matplotlib menyediakan lingkungan yang sangat efisien dan intuitif untuk implementasi teknik-teknik pengolahan citra. Setiap pustaka memiliki peran yang spesifik dan saling melengkapi dalam alur kerja pengolahan citra digital.

5.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar praktikum berikutnya mencakup percobaan dengan berbagai ukuran kernel yang berbeda untuk mengamati pengaruhnya terhadap hasil filtering. Selain itu, penambahan citra yang mengandung noise buatan (artificial noise) dapat memperjelas keunggulan masing-masing metode secara lebih terukur. Eksplorasi terhadap teknik-teknik filtering lanjutan seperti Gaussian filter atau bilateral filter juga dapat memperluas pemahaman tentang trade-off antara pengurangan noise dan pemeliharaan detail citra.